BÁO CÁO THỰC HÀNH LAB 18

PULL REQUEST

Người báo cáo: Liamz07

1. Mục tiêu:

- **Hiểu bản chất Pull Request:** Nắm được vai trò của PR như một cổng kiểm soát chất lượng (Quality Gate) trước khi đưa mã nguồn mới vào nhánh chính (main hoặc develop).
- **Thực hành quy trình khép kín:** Thành thạo chuỗi thao tác chuẩn từ môi trường cục bộ (Local) lên kho lưu trữ từ xa (Remote): feature → push → Pull Request → review → merge.
- **Kỹ năng Code Review:** Biết cách kiểm tra sự khác biệt mã nguồn (Files changed), để lại nhận xét (Comment) và phê duyệt (Approve) thay đổi.
- **Đồng bộ hóa kho chứa:** Cập nhật trạng thái mới nhất từ Remote về Local sau khi hoàn tất hợp nhất.

2. Môi trường và chuẩn bị:

- **Hệ điều hành:** Ubuntu Linux / macOS / Windows (sử dụng Terminal / Git Bash)
- **Công cụ và nền tảng:**
  - *Git CLI đã cấu hình thông tin định danh (user.name, user.email) và xác thực kết nối (SSH Key hoặc Personal Access Token).*
  - *Tài khoản GitHub và một kho chứa từ xa (Remote Repository) đã được liên kết với thư mục dự án trên máy cá nhân.*

3. Sơ đồ luồng công việc:

Toàn bộ bài thực hành tuân thủ chặt chẽ quy trình 5 bước tiêu chuẩn trong phát triển phần mềm nhóm:

1. Tạo nhánh (feature) → 2. Đẩy mã (push) → 3. Tạo Pull Request → 4. Kiểm duyệt (review) → 5. Hợp nhất (merge)

4. Nội dung thực hành chi tiết:

BÀI 1: KHỞI TẠO VÀ PHÁT TRIỂN TRÊN NHÁNH FEATURE (LOCAL)

- **Mục đích:** Cô lập tính năng mới đang phát triển để không làm ảnh hưởng đến sự ổn định của nhánh main.
- **Thao tác thực thi:**

```
mkdir git-conflict-lab
cd git-conflict-lab
git init

# Tạo tệp tin ban đầu
echo "Original content on main branch" > config.txt
git add config.txt
git commit -m "Initial commit with config.txt"
```

- **Kết quả:** Tính năng profile.js được đóng gói thành một commit hoàn chỉnh trên nhánh cục bộ feature/user-profile.

BÀI 2: ĐẨY NHÁNH TÍNH NĂNG LÊN KHO LƯU TRỮ TỪ XA (PUSH)

- **Mục đích:** Đưa mã nguồn từ máy cá nhân lên máy chủ GitHub để chia sẻ với các thành viên khác trong nhóm.
- **Thao tác thực thi:** git push -u origin feature/user-profile
- **Giải thích tham số:** Cờ -u (--set-upstream) giúp thiết lập liên kết theo dõi trực tiếp giữa nhánh feature/user-profile trên máy cục bộ và nhánh tương ứng trên origin (GitHub).
- **Kết quả:** Terminal báo thành công (* [new branch] feature/user-profile -> feature/user-profile).

BÀI 3: KHỞI TẠO PULL REQUEST TRÊN GITHUB

- **Mục đích:** Đề xuất yêu cầu hợp nhất mã nguồn từ nhánh tính năng (compare: feature/user-profile) vào nhánh gốc (base: main).
- **Thao tác thực thi trên giao diện Web (GitHub):**
  1. *Truy cập vào trang kho chứa (Repository) trên trình duyệt.*
  2. *Nhấn vào nút thông báo màu xanh "Compare & pull request" (hoặc chuyển sang tab Pull requests → nhấn New pull request).*
  3. *Kiểm tra chính xác hướng hợp nhất: base: main ← compare: feature/user-profile.*
  4. *Điền đầy đủ thông tin mô tả chuyên nghiệp:*
     - **Title:** [Feature] Thêm chức năng lấy thông tin hồ sơ người dùng (profile.js)
     - **Description:** Mô tả ngắn gọn mục đích thay đổi, các tệp tin đã thêm/sửa và cách kiểm thử.
  5. *Nhấn nút "Create pull request".*
- **Kết quả:** Cả hai nhánh main và feature/update-config đều đã tiến lên 1 commit mới, chỉnh sửa cùng một dòng trong tệp config.txt với nội dung hoàn toàn khác nhau.

BÀI 4: KIỂM DUYỆT MÃ NGUỒN (REVIEW)

- **Mục đích:** Đảm bảo mã nguồn tuân thủ quy chuẩn lập trình, không chứa lỗi logic hay lỗ hổng bảo mật trước khi đưa vào hệ thống chính.
- **Thao tác thực thi:**
  1. *Trong giao diện Pull Request vừa tạo, chuyển sang thẻ "Files changed" để xem chi tiết từng dòng mã được thêm (màu xanh) hoặc xóa (màu đỏ).*
  2. *Di chuột vào số thứ tự dòng code bất kỳ, nhấn dấu + màu xanh để viết nhận xét góp ý (Ví dụ: "Nên bổ sung kiểm tra điều kiện nếu id bị rỗng").*
  3. *Sau khi kiểm tra đạt yêu cầu, nhấn nút "Review changes" ở góc trên bên phải → chọn "Approve" (hoặc "Comment" nếu tự review PR của chính mình) → nhấn "Submit review".*

BÀI 5: HỢP NHẤT MÃ NGUỒN (MERGE) VÀ DỌN DẸP NHÁNH

- **Mục đích:** Chính thức tích hợp mã nguồn đã được phê duyệt vào nhánh main và làm sạch không gian làm việc.
- **Thao tác trên GitHub:**
  1. *Quay lại thẻ "Conversation" của Pull Request. Xác nhận trạng thái hiển thị "This branch has no conflicts with the base branch".*
  2. *Nhấn nút "Merge pull request" → nhấn "Confirm merge". Trạng thái PR chuyển sang màu tím với nhãn Merged.*
  3. *Nhấn nút "Delete branch" ngay bên dưới để xóa nhánh tạm feature/user-profile trên Remote, giữ cho danh sách nhánh luôn gọn gàng.*
- **Thao tác đồng bộ lại máy cục bộ (Local Cleanup):**

```
# Chuyển về nhánh main
git checkout main

# Kéo mã nguồn mới nhất (đã bao gồm tính năng vừa merge) về máy
git pull origin main

# Xóa nhánh tính năng cũ ở máy cục bộ
git branch -d feature/user-profile
```

5. Tổng hợp các bước và công cụ thực hành

Giai đoạn	Môi trường thực hiện	Lệnh / Thao tác chính	Mục đích & Ý nghĩa
Feature	Local Terminal	git checkout -b feature/<name>	Tạo không gian phát triển tính năng độc lập.
Push	Local → Remote	git push -u origin feature/<name>	Đưa các commit từ máy cá nhân lên GitHub.
Pull Request	GitHub Web UI	Compare & pull request	Mở yêu cầu xin phép gộp mã nguồn vào nhánh main.

Review	GitHub Web UI	Files changed → Review changes	Kiểm tra chéo chất lượng mã nguồn, phát hiện lỗi sớm.
Merge & Sync	GitHub & Local	Confirm merge & git pull origin main	Hợp nhất mã nguồn chính thức và đồng bộ lại máy cục bộ.

6. Kết luận và bài học rút ra:

- **Kết quả đạt được:** Hoàn thành trọn vẹn quy trình phát triển tính năng chuẩn chuyên nghiệp từ việc phân nhánh cục bộ (feature), đẩy mã (push), mở yêu cầu (Pull Request), kiểm duyệt (review) cho đến hợp nhất (merge) và dọn dẹp hệ thống.
- **Kỹ năng tích lũy:** Hiểu rõ tại sao các dự án phần mềm thực tế luôn khóa quyền đẩy trực tiếp (direct push) vào nhánh main và bắt buộc mọi thay đổi phải đi qua Pull Request — đây là cơ chế cốt lõi giúp duy trì tính ổn định của sản phẩm và nâng cao chất lượng code thông qua làm việc nhóm.

HẾT!